SIMATS ENGINEERING

CO2_AT3 Error Analysis Task

COURSE NAME: **Software Testing** **COURSE CODE: CSA37**

COURSE OUTCOME COVERED

CO2: *Design effective test cases using black-box, white-box testing techniques and to control methods flow.*

Assessment Tool Weightage: 20%

Sl. No	Questions	Bloom' s Level
1	A tester designs test cases for an online shopping system using equivalence partitioning but incorrectly groups input ranges (e.g., valid and invalid inputs mixed in one class). Additionally, boundary values are not considered. Analyze the given test design and identify all errors in equivalence class partitioning and boundary value analysis. Examine the root causes of these mistakes with proper reasoning. Propose corrected test cases with proper classification and boundary coverage. Justify how your corrections improve defect detection effectiveness.	3
2	A QA engineer performs random testing on a banking application and claims high coverage without using any structured techniques. Critical defects in transaction limits and boundary conditions are missed. Evaluate the testing approach and identify errors in strategy and coverage assumptions. Analyze the root causes behind missing critical defects. Suggest appropriate black box testing techniques to correct the approach. Justify how these techniques ensure better test adequacy and reliability.	3
3	A developer performs white box testing but creates an incomplete control flow graph, missing decision nodes and loops, resulting in poor path coverage. Analyze the errors in the control flow graph and path testing approach. Identify the root causes for inadequate test coverage. Propose corrections by constructing a proper control flow representation and identifying all independent paths. Justify how improved path testing ensures better coverage and defect detection.	5
4	A testing team reports 100% statement coverage for a module but fails to detect logical errors in conditional statements. Evaluate the adequacy of the testing performed and identify gaps in coverage criteria. Analyze why statement coverage alone was insufficient. Recommend improved white box testing strategies such as branch or path coverage. Justify	5
	how the revised approach enhances overall test	

	effectiveness and error detection.	
5	A tester uses Equivalence Class Partitioning for an age-validation field (0-100) but defines only two classes: Negative numbers and Positive numbers. Analyze the errors in this partitioning strategy. Identify why this fails to account for valid vs. invalid age limits and the absence of Boundary Value Analysis. Propose a corrected set of partitions and justify how they ensure more robust input validation.	3
6	A developer claims full Condition Coverage for a complex `if(A && B	
7	A QA team applies Decision Table Testing for a discount system but misses several combinations of "Premium Member" and "Coupon Applied," leading to price calculation errors. Analyze the errors in the construction of the decision table. Identify why the current approach is incomplete. Propose a revised, exhaustive decision table and justify its effectiveness in preventing production defects.	3
8	During unit testing of a looping structure, a tester checks only the first and last iterations (n=1 and n=max). Defects occurring at n=0 and n=max+1 are missed. Evaluate the errors in this Loop Testing strategy. Analyze the root causes behind these omissions. Recommend a corrected approach for testing simple and nested loops, and justify how it ensures path integrity.	5
9	An engineer utilizes Error Guessing as the primary testing technique for a safety-critical braking system, ignoring structured black box methods. Identify the errors in relying solely on ad-hoc techniques for high-risk modules. Analyze the impact on test adequacy. Suggest a combination of BVA and State Transition Testing to fix the strategy, justifying the increased safety assurance.	5
10	A tester designs a Control Flow Graph (CFG) for a module but fails to represent the "else" branches of multiple nested if statements, leading to 100% statement coverage but 0% branch coverage on error paths. Analyze the errors in graph construction. Identify the root causes for the missing nodes. Propose a corrected CFG and independent paths to ensure branch adequacy.	5

Question 1 – Equivalence Partitioning and BVA in an Online Shopping System

Assumed scenario: The "Quantity" field of the cart accepts integers from 1 to 10 per item. The tester built the following flawed design.

Flawed class	Input values used	Problem
Class A: "Quantity 0–5"	0, 3, 5	Mixes invalid (0) with valid (3, 5)
Class B: "Quantity 6–15"	6, 12, 15	Mixes valid (6–10) with invalid (11–15)
No class	—	Negative, non-numeric, empty and decimal inputs never partitioned

Errors identified

- Classes are not homogeneous: valid and invalid inputs share a class, so one representative value cannot stand for the whole class.
- Invalid classes are missing (negative, non-integer, alphabetic, special characters, null/blank).
- Boundary values 0, 1, 10 and 11 were not selected deliberately; 0 only appears by accident.
- Test cases do not record expected results per class (accept / reject with error message).

Root causes

- Partitions were drawn by arbitrary numeric ranges rather than by the specification's valid/invalid behaviour.
- Misunderstanding of the EP principle: every member of a class must be treated identically by the system.
- BVA treated as optional, though defects cluster at edges (off-by-one errors such as < vs <=).
- No requirement-to-test traceability, so data-type and empty-input rules were overlooked.

Corrected design

Class	Type	Range	Representative
EC1	Valid	1 ≤ Qty ≤ 10	5
EC2	Invalid	Qty < 1 (zero/negative)	-3
EC3	Invalid	Qty > 10	15
EC4	Invalid	Non-integer (decimal)	2.5
EC5	Invalid	Non-numeric	"abc", "@#"
EC6	Invalid	Empty / null	(blank)

TC	Input	Technique	Expected result
TC1	5	EP (EC1)	Accepted
TC2	-3	EP (EC2)	Rejected – error
TC3	15	EP (EC3)	Rejected – max limit message
TC4	2.5	EP (EC4)	Rejected
TC5	abc	EP (EC5)	Rejected
TC6	blank	EP (EC6)	Rejected – required field

Class	Type	Range	Representative
TC7	0	BVA (min−1)	Rejected
TC8	1	BVA (min)	Accepted
TC9	2	BVA (min+1)	Accepted
TC10	9	BVA (max−1)	Accepted
TC11	10	BVA (max)	Accepted
TC12	11	BVA (max+1)	Rejected

Justification

Homogeneous classes guarantee that one test per class represents the whole class, removing redundancy and gaps. Boundary tests (0, 1, 10, 11) directly expose off-by-one and relational-operator defects, which are the most frequent input-validation faults. Adding invalid data-type classes detects crashes and unhandled exceptions. Together, 12 targeted tests give far higher defect-detection effectiveness than the original 6 loosely chosen values.

Question 2 – Random Testing of a Banking Application

Errors in strategy and coverage assumptions

- "High coverage" was claimed without any coverage metric – coverage was assumed from the number of tests executed, not measured against requirements or code.
- Inputs were chosen at random, so there is no guarantee that limit values (e.g., daily limit ₹50,000, minimum balance, per-transaction cap) were ever exercised.
- No test-oracle or expected-result design, so some failures may have passed unnoticed.
- Tests are not repeatable or traceable, hence cannot be reviewed or regression-tested.
- Business-rule combinations (account type × amount × channel) were never systematically explored.

Root causes of missed defects

- Boundary regions occupy a very small fraction of the input domain; random selection has a very low probability of hitting exactly limit, limit+1 or limit−1.
- No analysis of the specification, so critical rules (limits, state of account) were unknown to the tester.
- Overconfidence bias: volume of tests mistaken for quality of tests.
- Absence of risk-based prioritisation for high-impact financial functions.

Recommended black-box techniques

Technique	Application to banking
Equivalence Partitioning	Classes: amount ≤ 0, 1–50,000 (valid), > 50,000; account active / dormant / blocked
Boundary Value Analysis	Test 0, 1, 49,999, 50,000, 50,001 against the transaction limit; balance = minimum, minimum−1
Decision Table Testing	Combine balance sufficient, daily limit exceeded, account status, OTP verified → approve / reject
State Transition Testing	Account states (Active → Locked after 3 failed PINs → Unlocked)
Use-case / Error guessing (supplementary)	Concurrent transfers, duplicate submission, session timeout

Justification

Structured techniques derive tests from the specification, so each requirement and each boundary is covered by design and can be traced and measured (requirement coverage, partition coverage, boundary coverage). Boundary and decision-based tests concentrate effort where defects cluster, giving test adequacy that random testing cannot demonstrate. Repeatable, documented test cases also improve reliability by supporting regression testing after every change.

Question 3 – Incomplete Control Flow Graph and Path Testing

Assumed module under test:

```
1  sum = 0; neg = 0; i = 1;
2  while (i <= n) {
3     if (a[i] > 0)
4        sum = sum + a[i];
5     else if (a[i] < 0)
6        neg = neg + 1;
7     i = i + 1;
   }
8  if (neg > 0)
9     print "Warning";
   else
10    print "OK";
11 end
```

Errors in the developer's CFG and path approach

- Loop condition node (2) and the back edge 7→2 were omitted, so the CFG looks like a straight sequence and loop behaviour was never tested.
- The decision nodes 3 and 5 were merged into a single statement, hiding the else-if (zero element) path.
- The final decision node 8 and its false branch (10) were missing.
- Only one end-to-end path was tested, so path and branch coverage stayed very low.
- Cyclomatic complexity V(G) was not computed, so the number of required tests was unknown.

Root causes

- CFG drawn from the programmer's intent ("happy path") rather than from the actual code structure.
- Compound constructs (while, if-else-if) not decomposed into basic blocks and predicate nodes.
- No formal procedure (nodes → edges → V(G) → basis paths) was followed.
- Inadequate code review of the graph against the source.

Corrected CFG (nodes and edges)

Node	Meaning	Successors
1	Initialisation (lines 1)	2
2	Predicate: i <= n	3 (T), 8 (F)
3	Predicate: a[i] > 0	4 (T), 5 (F)
4	sum = sum + a[i]	7
5	Predicate: a[i] < 0	6 (T), 7 (F)
6	neg = neg + 1	7
7	i = i + 1	2 (back edge)
8	Predicate: neg > 0	9 (T), 10 (F)

Node	Meaning	Successors
9	print "Warning"	11
10	print "OK"	11
11	End	—

Cyclomatic complexity: E = 14, N = 11 → V(G) = E − N + 2 = 14 − 11 + 2 = **5**. Check by predicate nodes: 4 predicates (2, 3, 5, 8) + 1 = 5.

Independent (basis) paths with test data

Path	Node sequence	Test data	Expected
P1	1-2-8-10-11	n = 0	OK (loop skipped)
P2	1-2-3-4-7-2-8-10-11	n = 1, a = {5}	OK
P3	1-2-3-5-6-7-2-8-9-11	n = 1, a = {-4}	Warning
P4	1-2-3-5-7-2-8-10-11	n = 1, a = {0}	OK
P5	1-2-3-4-7-2-3-5-6-7-2-8-9-11	n = 2, a = {5, -4}	Warning (loop repeated)

Justification

A complete CFG exposes every decision outcome and the loop, and V(G) = 5 gives the minimum number of independent paths to execute. Executing all five paths covers every statement and every branch (all 14 edges) at least once, and the P5 path exercises the loop back edge with state carried between iterations. This detects faults in loop control, conditions and data-flow that a single-path test cannot reveal.

Question 4 – 100% Statement Coverage but Undetected Logical Errors

Illustration:

```
if (age >= 18 && hasId)      // intended condition
   allowEntry = true;
else
   allowEntry = false;
// Suppose coded as:  if (age >= 18 || hasId)
```

A single test (age = 20, hasId = true) executes every statement of the true branch only if no else statement existed; in general, a test set such as {age=20, hasId=true} plus {age=10, hasId=false} gives 100% statement coverage yet never separates && from ||.

Adequacy evaluation and gaps

- Statement coverage only proves that each line ran at least once; it says nothing about whether each decision was evaluated both true and false.
- Missing coverage criteria: branch (decision) coverage, condition coverage, MC/DC and path coverage.
- Implicit else paths (no statements) contribute nothing to statement coverage, so they remain untested.
- The report treated a coverage percentage as proof of correctness.

Why statement coverage alone was insufficient

- Logical errors (wrong operator, inverted condition, wrong boundary like > instead of >=) change which branch is taken, not whether a statement executes.
- Combinations of conditions and loops produce many paths that statements-covering tests skip.
- Coverage is a necessary, not a sufficient, adequacy measure.

Recommended improved strategy

Criterion	What it demands	Example result
Branch coverage	Every decision both TRUE and FALSE	Adds a test with age = 10, hasId = true to expose the || fault
Condition coverage	Each atomic condition both T and F	age ≥ 18 and hasId each T/F
MC/DC	Each condition independently affects outcome	(T,T),(F,T),(T,F) – 3 tests for 2 conditions
Path coverage / basis path	Independent paths via V(G)	Used for loops and nested decisions

Justification

Branch coverage subsumes statement coverage and forces both outcomes of each decision, so wrong operators and inverted conditions produce observable failures. MC/DC adds rigorous checking of compound predicates with only n+1 tests for n conditions. Combining these with a mutation-style review of conditions raises both coverage depth and fault-detection capability while remaining affordable.

Question 5 – Age Validation Field (0–100)

Errors in the partitioning

- Only two classes (negative, positive) were used; "positive" mixes valid ages (0–100) with invalid ages (> 100), so the class is not homogeneous.
- Zero is neither clearly negative nor positive and was left unclassified.
- No invalid non-numeric, decimal or blank classes.
- BVA absent: 0, 100, −1 and 101 – the likely fault points – were never tested.

Root causes

- Partitions were based on the sign of the number, not on the specified valid range 0–100.
- The tester ignored the requirement's upper limit and the data-type constraint.
- Misapplication of EP: classes must reflect distinct system behaviour (accept vs reject).

Corrected partitions and tests

Class	Type	Range	Test input	Expected
EC1	Valid	0 ≤ age ≤ 100	45	Accept
EC2	Invalid	age < 0	-5	Reject
EC3	Invalid	age > 100	150	Reject
EC4	Invalid	Decimal	25.5	Reject
EC5	Invalid	Non-numeric	"abc"	Reject
EC6	Invalid	Blank	(empty)	Reject

BVA test	Input	Expected
Min − 1	-1	Reject
Min	0	Accept
Min + 1	1	Accept
Max − 1	99	Accept
Max	100	Accept
Max + 1	101	Reject

Justification

The revised partitions separate every distinct behaviour (accept, out-of-range low/high, wrong type, missing) so each class is homogeneous. BVA targets the exact edges where relational-operator faults (e.g., age < 100 instead of <= 100) hide. The result is robust validation with just 12 tests.

Question 6 – Condition Coverage for a Complex Decision

Note: The question text in the uploaded file is cut off after "if(A && B". It is assumed to be **if (A && B || C)**; the same reasoning applies to any compound predicate.

Errors in the developer's claim

- Condition coverage only requires each atomic condition (A, B, C) to be TRUE and FALSE at least once; it does not require the whole decision to take both outcomes.
- Example: tests (A,B,C) = (T,F,T) and (F,T,F) make every condition take both values, yet the first gives overall TRUE and the second FALSE – a weak set that never shows B or A alone affecting the result. Another set, (T,F,F) and (F,T,F), covers all conditions both ways but the decision is FALSE both times, so the THEN branch is never executed.
- Short-circuit evaluation means some conditions (e.g., B when A is false) are never evaluated at all, so claiming they were "covered" is false.
- Condition coverage does not subsume branch coverage.

Root causes

- Confusion between condition coverage and decision coverage.
- Ignoring short-circuit semantics of && and ||.
- Selecting data to satisfy the metric superficially, not to expose logic faults.

Correction – Condition/Decision and MC/DC

Test	A	B	C	Outcome	Purpose
T1	T	T	F	TRUE	Baseline
T2	F	T	F	FALSE	Flip A → outcome flips (A independent)
T3	T	F	F	FALSE	Flip B → outcome flips (B independent)
T4	F	F	T	TRUE	C true forces TRUE
T5	F	F	F	FALSE	Flip C → outcome flips (C independent)

T1/T2 show A's independent effect, T1/T3 show B's, and T4/T5 show C's. Four to five tests (n+1 to 2n) therefore achieve MC/DC, full condition coverage and full decision coverage.

Justification

MC/DC proves that each condition can independently change the decision outcome, which detects operator mistakes (&& vs ||), swapped variables and masked conditions that plain condition coverage misses, while avoiding the 2^n blow-up of multiple-condition coverage (8 combinations here).

Question 7 – Decision Table Testing for a Discount System

Assumed rules: Premium members get 10%; a valid coupon gives a further 5%; orders of ₹1000 or more get free delivery. Conditions: Premium Member (P), Coupon Applied (C), Order ≥ ₹1000 (O).

Errors in the construction of the table

- Only some combinations of P and C were listed (e.g., P=Y,C=Y and P=N,C=N); P=Y,C=N and P=N,C=Y were omitted.
- The third condition (order value) was not combined with the others, so 2^n rows (here 8) were not enumerated.
- No check for impossible or redundant columns, and no actions assigned to every rule.
- Coupon-stacking rule (premium + coupon: additive or multiplicative) was not defined.

Root causes

- Rules were written from the "typical" cases a user thinks of, not systematically from the conditions.
- The 2^n formula was not used to verify completeness.
- Requirement ambiguity about stacking was not resolved before test design.

Revised exhaustive decision table ($2^3 = 8$ rules)

Rule	Premium	Coupon	Order ≥ 1000	Discount	Delivery
R1	Y	Y	Y	10% + 5%	Free
R2	Y	Y	N	10% + 5%	Charged
R3	Y	N	Y	10%	Free
R4	Y	N	N	10%	Charged
R5	N	Y	Y	5%	Free
R6	N	Y	N	5%	Charged
R7	N	N	Y	0%	Free
R8	N	N	N	0%	Charged

One test case is written per rule; boundary order values 999 and 1000 can be added for the "Order ≥ 1000" condition.

Justification

Because all 8 combinations are listed, no logical case is left unverified, in particular the previously missing Premium-only and Coupon-only rules that caused price errors. Each rule maps to an expected price, giving an unambiguous oracle and making omissions and contradictions in requirements visible before release, which prevents production pricing defects and financial loss.

Question 8 – Loop Testing Strategy

Errors in the strategy

- Only n = 1 and n = max were checked; the zero-iteration case (n = 0) was not tested.
- Out-of-range iteration n = max + 1 was not tested, so loop-termination and array-overrun faults were missed.
- Typical iteration (n = 2 or a mid value) and n = max − 1 were skipped.
- Nested loops were not considered at all.

Root causes

- Tester assumed that testing the "extremes" of valid iterations is sufficient, ignoring off-by-one errors in loop conditions (<, <=).
- Unawareness that a loop can be skipped entirely, or that the bound itself may be violated.
- No structured loop-testing checklist.

Corrected approach – simple loops (maximum n passes)

Test	Iterations	Defect targeted
1	0 (skip loop)	Initialisation errors, loop never entered
2	1	Single-pass behaviour
3	2	Repeat / state carried across passes
4	m (1 < m < n)	Typical operation
5	n − 1	Almost-full loop
6	n	Upper bound handled
7	n + 1	Off-by-one, overflow, missing termination

Corrected approach – nested loops (Beizer)

- Set all outer loops to minimum iterations and test the innermost loop as a simple loop.
- Move outward one loop at a time: test each outer loop with the simple-loop tests while inner loops stay at minimum values.
- Finally, test with all loops at typical and then maximum values together.
- For concatenated loops with dependent counters, treat them as nested.

Justification

Testing zero, one, typical, max−1, max and max+1 iterations exercises every way control can enter, repeat and leave a loop, so the paths through the loop condition and body are all verified (path integrity). The nested-loop method reduces test growth from n^k to roughly k·n tests while still checking interactions between loop levels.

Question 9 – Error Guessing for a Safety-Critical Braking System

Errors in relying on error guessing

- Error guessing is experience-based and informal: it has no coverage measure, so test adequacy cannot be demonstrated for certification (e.g., ISO 26262).
- Systematic input-boundary conditions (speed, pedal pressure, wheel-slip thresholds) were ignored.
- State-dependent behaviour (normal, ABS active, fault/limp mode) was not modelled.
- Results are not reproducible and depend on one engineer's knowledge.

Impact on test adequacy

- Large unexplored input and state space → latent catastrophic failures.
- No traceability from requirements to tests, so audit and safety case are incomplete.
- False confidence: absence of found defects is taken as absence of defects.

Proposed strategy: BVA + State Transition Testing

BVA on inputs (e.g., ABS engages at wheel slip > 20%, brake pedal input 0–100%):

Parameter	Boundary tests
Pedal pressure (0–100%)	−1, 0, 1, 99, 100, 101
Wheel slip threshold 20%	19, 20, 21
Vehicle speed trigger (e.g., 5 km/h)	4, 5, 6
Sensor response time limit	limit−1 ms, limit, limit+1 ms

State transition testing:

Current state	Event	Next state	Test goal
Idle	Pedal pressed	Braking	Normal braking engaged
Braking	Slip > 20%	ABS Active	ABS triggers at boundary
ABS Active	Slip ≤ 20%	Braking	Returns correctly
Braking / ABS	Pedal released	Idle	Brake released fully
Any	Sensor fault	Fail-safe mode	Warning + safe degraded braking
Idle	Invalid event (slip signal)	Idle	Invalid transition rejected

Coverage target: all states, all valid transitions (0-switch) and invalid transitions, plus transition pairs (1-switch) for critical sequences.

Justification

BVA catches threshold faults at the exact limits that trigger safety functions, and state-transition tests verify that the system moves correctly (and safely) between modes, including faults and illegal events. Both give measurable coverage and traceability required for safety certification, while error guessing is retained only as a supplement to add experience-based cases such as sensor dropout or rapid pedal pumping.

Question 10 – CFG Missing "Else" Branches in Nested Ifs

Assumed module:

```
1  read(x, y);
2  if (x > 0) {
3     if (y > 0)
4        result = x + y;
5     else
6        error("y invalid");
7  } else
8     error("x invalid");
9  print(result);
```

Errors in graph construction

- The else edges from decision nodes (x > 0) and (y > 0) to the error nodes were not drawn; every decision showed only a TRUE outgoing edge.
- Error-handling nodes 6 and 8 were missing from the graph.
- Missing join (merge) node before print(result).
- Because only the "if" side existed, one test (x>0, y>0) covered all drawn nodes (100% statement coverage in the graph) but 0% of the error branches.

Root causes

- Else paths were considered "exceptional" and omitted; the graph was drawn from the success scenario.
- Each decision node was not checked for exactly two outgoing edges (T/F).
- No cross-check between the CFG and the code, and no branch-coverage requirement stated.

Corrected CFG

Node	Description	Edges
N1	read(x, y)	→ N2
N2	Decision: x > 0	T → N3, F → N5
N3	Decision: y > 0	T → N4, F → N6
N4	result = x + y	→ N7
N5	error("x invalid")	→ N7
N6	error("y invalid")	→ N7
N7	print(result) (join)	→ End

E = 8 (N1-N2, N2-N3, N2-N5, N3-N4, N3-N6, N4-N7, N5-N7, N6-N7) plus N7-End = 9; N = 8 including End. V(G) = 9 − 8 + 2 = **3** (predicates 2 + 1).

Independent paths for branch adequacy

Path	Route	Test data	Expected
P1	N1-N2-N3-N4-N7	x = 5, y = 3	result = 8
P2	N1-N2-N3-N6-N7	x = 5, y = -2	"y invalid" error
P3	N1-N2-N5-N7	x = -1, y = 3	"x invalid" error

Justification

Each decision now has both T and F edges, so executing P1–P3 covers all 4 decision outcomes (100% branch coverage) as well as all statements, including the error-handling code that carries the greatest risk. The count of independent paths equals V(G) = 3, confirming the test set is both necessary and sufficient for basis-path adequacy.

Code of Conduct:

I M Ravi Sai Vinay (192311035) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: M Ravi Sai Vinay

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Error Identification	20%	Accurately identifies all errors with clear understanding of issues	Identifies most errors with minor omissions	Identifies limited errors; some are missed	Fails to identify key errors
Root Cause Analysis	30%	Clearly explains underlying causes with strong technical reasoning	Explains causes with moderate clarity	Partial explanation; weak reasoning	Unable to identify causes of errors
Correction & Justification	25%	Provides correct solutions with strong justification and logic	Provides correct corrections with some justification	Corrections are partially correct or weakly justified	Incorrect or no corrections provided
Application of Concepts	15%	Effectively applies relevant concepts to analyze and resolve errors	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts
Clarity & Presentation	10%	Presents analysis clearly, logically, and systematically	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand